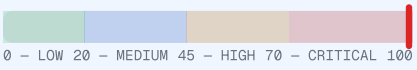
SECURITY VULNERABILITY REPORT

Static Analysis & Threat Assessment

Repository: OmarRao/analyzer

Branch: main · Analysis Engine: Semgrep + MITRE ATT&CK Mapping

100
/ 100
CRITICAL
Composite Risk Score



SCORING BREAKDOWN

Critical findings × 10	+2450
Warning findings × 3	+507
Dependency CVEs × 8	+0
Capped at	100

245	169	0	10
Critical Findings	Warnings	Dependency CVEs	CWE Categories

SecureScope · Open Source Security Analysis Platform
Findings mapped to MITRE ATT&CK v14 · CWE Top 25 · OWASP Top 10

Omar Rao
Engineer by trait · Data Resilience, Privacy & Cybersecurity Expert
Founder, SecureScope

EXECUTIVE SUMMARY

Scan Overview

REPOSITORY	
Owner	OmarRao
Repository	analyzer
Branch	main
Analysis Engine	Semgrep
Generated	June 11, 2026 at 09:33 UTC

FINDING BREAKDOWN	
Total Findings	414
Critical (ERROR)	245
Warning	169
Dependency CVEs	0
CWE Categories Hit	10

RISK SCORE DETAIL

Composite Risk Assessment

RISK SCORE

100/100

THREAT GRADE

CRITICAL

Score calculated as: `(critical × 10) + (warnings × 3) + (CVEs × 8) + (sandbox behaviors × 15)`, capped at 100. A score of 70+ is rated CRITICAL and indicates immediate remediation is required.

CWE DISTRIBUTION

Vulnerability Category Breakdown

SecureScope Security Report · OmarRao/analyzer Page 2 of 62

CWE IDENTIFIER	COUNT	SHARE	DISTRIBUTION
CWE-704	116	28%	
CWE-78	103	25%	
CWE-89	97	23%	
CWE-79	35	8%	
CWE-327	33	8%	
CWE-918	24	6%	
CWE-22	3	1%	
CWE-250	1	0%	
CWE-668	1	0%	
CWE-489	1	0%	

SecureScope Security Report · OmarRao/analyzer Page 3 of 62

MITRE ATT&CK MAPPING

Attack Surface Analysis

Each vulnerability class is mapped to the corresponding MITRE ATT&CK technique and tactic. Counts reflect findings from the static analysis phase. Exposed vectors require immediate remediation per the AI Fix Advisor output.

VULNERABILITY TYPE	CWE	ATT&CK TECHNIQUE	STATUS
SQL Injection	CWE-89	T1190	97 findings
Command Injection	CWE-78	T1059	103 findings
Cross-Site Scripting	CWE-79	T1059.007	35 findings
SSRF	CWE-918	T1090	24 findings
Path Traversal	CWE-22	T1083	3 findings
Hardcoded Credentials	CWE-798	T1552.001	Clear
Weak Cryptography	CWE-327	T1600	33 findings
Insecure Deserialization	CWE-502	T1059	Clear

TOP PRIORITY FINDINGS

Critical Issues Requiring Immediate Action

Missing User

CRITICAL

Dockerfile · line 12

By not specifying a USER, a program in the container may run as 'root'. This is a security hazard. If an attacker can control a process running as root, they may have control over the container. Ensure that the last USER

CWE-250

Tainted Sql String

CRITICAL

api\accounts.py · line 30

Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction could accidentally result in a SQL injection. An attacker could use a SQL injection to steal or modify

CWE-704

Tainted Sql String

CRITICAL

api\accounts.py · line 42

Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction could accidentally result in a SQL injection. An attacker could use a SQL injection to steal or modify

CWE-704

SecureScope Security Report · OmarRao/analyzer Page 4 of 62

Tainted Sql String

CRITICAL

api\accounts.py · line 71

Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction could accidentally result in a SQL injection. An attacker could use a SQL injection to steal or modify

CWE-704

Tainted Sql String

CRITICAL

api\accounts.py · line 81

Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction could accidentally result in a SQL injection. An attacker could use a SQL injection to steal or modify

CWE-704

Tainted Sql String

CRITICAL

api\accounts.py · line 94

Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction could accidentally result in a SQL injection. An attacker could use a SQL injection to steal or modify

CWE-704

Tainted Sql String

CRITICAL

api\accounts.py · line 103

Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction could accidentally result in a SQL injection. An attacker could use a SQL injection to steal or modify

CWE-704

Tainted Sql String

CRITICAL

api\accounts.py · line 112

Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction could accidentally result in a SQL injection. An attacker could use a SQL injection to steal or modify

CWE-704

SecureScope Security Report · OmarRao/analyzer Page 5 of 62

Complete Findings Table (414 findings)

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
CRITICAL	**Missing User**	Dockerfile:12	CWE-250		By not specifying a USER, a program in the container may run as 'root'. This is a security hazard. If an attacker can co
CRITICAL	**Tainted Sql String**	api\accounts.py:30	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\accounts.py:42	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\accounts.py:71	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\accounts.py:81	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\accounts.py:94	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\accounts.py:103	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\accounts.py:112	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co

SecureScope Security Report · OmarRao/analyzer Page 6 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
CRITICAL	**Subprocess Injection**	api\accounts.py:123	CWE-78	T1059	Detected user input entering a `subprocess` call unsafely. This could result in a command injection vulnerability. An at
CRITICAL	**Dangerous Subprocess Use**	api\accounts.py:124	CWE-78	T1059	Detected subprocess function 'account_statement' with user controlled data. A malicious actor could leverage this to per
CRITICAL	**Dangerous Subprocess Use**	api\accounts.py:124	CWE-78	T1059	Detected subprocess function 'check_output' with user controlled data. A malicious actor could leverage this to perform
CRITICAL	**Subprocess Shell True**	api\accounts.py:125	CWE-78	T1059	Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn the command
CRITICAL	**Tainted Sql String**	api\accounts.py:145	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\accounts.py:158	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\accounts.py:162	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\accounts.py:180	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co

SecureScope Security Report · OmarRao/analyzer Page 7 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
CRITICAL	**Tainted Sql String**	api\accounts.py:189	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Ssrf Injection Urllib**	api\accounts.py:197	CWE-918	T1090	Data from request object is passed to a new server-side request. This could lead to a server-side request forgery (SSRF)
CRITICAL	**Tainted Sql String**	api\accounts.py:203	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Dangerous System Call**	api\accounts.py:213	CWE-78	T1059	Found user-controlled data used in a system call. This could allow a malicious actor to execute commands. Use the 'subpr
CRITICAL	**Subprocess Injection**	api\accounts.py:223	CWE-78	T1059	Detected user input entering a `subprocess` call unsafely. This could result in a command injection vulnerability. An at
CRITICAL	**Dangerous Subprocess Use**	api\accounts.py:224	CWE-78	T1059	Detected subprocess function 'check_output' with user controlled data. A malicious actor could leverage this to perform
CRITICAL	**Subprocess Shell True**	api\accounts.py:225	CWE-78	T1059	Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn the command
CRITICAL	**Tainted Sql String**	api\accounts.py:236	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co

SecureScope Security Report · OmarRao/analyzer Page 8 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
CRITICAL	**Tainted Sql String**	api\accounts.py:248	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\admin.py:50	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\admin.py:60	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\admin.py:71	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Subprocess Injection**	api\admin.py:79	CWE-78	T1059	Detected user input entering a `subprocess` call unsafely. This could result in a command injection vulnerability. An at
CRITICAL	**Dangerous Subprocess Use**	api\admin.py:79	CWE-78	T1059	Detected subprocess function 'check_output' with user controlled data. A malicious actor could leverage this to perform
CRITICAL	**Subprocess Shell True**	api\admin.py:79	CWE-78	T1059	Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn the command
CRITICAL	**Dangerous System Call**	api\admin.py:87	CWE-78	T1059	Found user-controlled data used in a system call. This could allow a malicious actor to execute commands. Use the 'subpr

SecureScope Security Report · OmarRao/analyzer Page 9 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
CRITICAL	**Subprocess Injection**	api\admin.py:97	CWE-78	T1059	Detected user input entering a `subprocess` call unsafely. This could result in a command injection vulnerability. An at
CRITICAL	**Dangerous Subprocess Use**	api\admin.py:97	CWE-78	T1059	Detected subprocess function 'check_output' with user controlled data. A malicious actor could leverage this to perform
CRITICAL	**Subprocess Shell True**	api\admin.py:97	CWE-78	T1059	Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn the command
CRITICAL	**Subprocess Injection**	api\admin.py:106	CWE-78	T1059	Detected user input entering a `subprocess` call unsafely. This could result in a command injection vulnerability. An at
CRITICAL	**Dangerous Subprocess Use**	api\admin.py:107	CWE-78	T1059	Detected subprocess function 'check_output' with user controlled data. A malicious actor could leverage this to perform
CRITICAL	**Subprocess Shell True**	api\admin.py:107	CWE-78	T1059	Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn the command
CRITICAL	**Tainted Sql String**	api\admin.py:125	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Path Traversal Open**	api\admin.py:136	CWE-22	T1083	Found request data in a call to 'open'. Ensure the request data is validated or sanitized, otherwise it could result in

SecureScope Security Report · OmarRao/analyzer Page 10 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
CRITICAL	**Tainted Sql String**	api\admin.py:171	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\admin.py:174	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Subprocess Injection**	api\admin.py:186	CWE-78	T1059	Detected user input entering a `subprocess` call unsafely. This could result in a command injection vulnerability. An at
CRITICAL	**Dangerous Subprocess Use**	api\admin.py:187	CWE-78	T1059	Detected subprocess function 'check_output' with user controlled data. A malicious actor could leverage this to perform
CRITICAL	**Subprocess Shell True**	api\admin.py:187	CWE-78	T1059	Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn the command
CRITICAL	**Tainted Sql String**	api\admin.py:206	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Dangerous System Call**	api\admin.py:215	CWE-78	T1059	Found user-controlled data used in a system call. This could allow a malicious actor to execute commands. Use the 'subpr
CRITICAL	**Subprocess Injection**	api\admin.py:223	CWE-78	T1059	Detected user input entering a `subprocess` call unsafely. This could result in a command injection vulnerability. An at

SecureScope Security Report · OmarRao/analyzer Page 11 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
CRITICAL	**Dangerous Subprocess Use**	api\admin.py:224	CWE-78	T1059	Detected subprocess function 'check_output' with user controlled data. A malicious actor could leverage this to perform
CRITICAL	**Subprocess Shell True**	api\admin.py:224	CWE-78	T1059	Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn the command
CRITICAL	**Tainted Sql String**	api\admin.py:254	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Subprocess Injection**	api\admin.py:265	CWE-78	T1059	Detected user input entering a `subprocess` call unsafely. This could result in a command injection vulnerability. An at
CRITICAL	**Dangerous Subprocess Use**	api\admin.py:266	CWE-78	T1059	Detected subprocess function 'check_output' with user controlled data. A malicious actor could leverage this to perform
CRITICAL	**Subprocess Shell True**	api\admin.py:266	CWE-78	T1059	Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn the command
CRITICAL	**Tainted Sql String**	api\auth.py:36	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\auth.py:42	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co

SecureScope Security Report · OmarRao/analyzer Page 12 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
CRITICAL	**Tainted Sql String**	api\auth.py:63	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\auth.py:87	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\auth.py:90	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\auth.py:102	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\auth.py:116	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\auth.py:121	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\auth.py:135	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\auth.py:139	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co

SecureScope Security Report · OmarRao/analyzer Page 13 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
CRITICAL	**Tainted Sql String**	api\auth.py:149	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Ssrf Injection Urllib**	api\auth.py:157	CWE-918	T1090	Data from request object is passed to a new server-side request. This could lead to a server-side request forgery (SSRF)
CRITICAL	**Tainted Sql String**	api\auth.py:178	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\auth.py:191	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\auth.py:194	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\auth.py:208	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\auth.py:218	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\auth.py:221	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co

SecureScope Security Report · OmarRao/analyzer Page 14 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
CRITICAL	**Tainted Sql String**	api\auth.py:233	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\files.py:59	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\files.py:69	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Subprocess Injection**	api\files.py:74	CWE-78	T1059	Detected user input entering a `subprocess` call unsafely. This could result in a command injection vulnerability. An at
CRITICAL	**Dangerous Subprocess Use**	api\files.py:74	CWE-78	T1059	Detected subprocess function 'delete_file' with user controlled data. A malicious actor could leverage this to perform c
CRITICAL	**Subprocess Shell True**	api\files.py:74	CWE-78	T1059	Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn the command
CRITICAL	**Tainted Sql String**	api\files.py:75	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\files.py:86	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co

SecureScope Security Report · OmarRao/analyzer Page 15 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
CRITICAL	**Tainted Sql String**	api\files.py:101	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Subprocess Injection**	api\files.py:123	CWE-78	T1059	Detected user input entering a `subprocess` call unsafely. This could result in a command injection vulnerability. An at
CRITICAL	**Dangerous Subprocess Use**	api\files.py:124	CWE-78	T1059	Detected subprocess function 'check_output' with user controlled data. A malicious actor could leverage this to perform
CRITICAL	**Subprocess Shell True**	api\files.py:124	CWE-78	T1059	Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn the command
CRITICAL	**Subprocess Injection**	api\files.py:134	CWE-78	T1059	Detected user input entering a `subprocess` call unsafely. This could result in a command injection vulnerability. An at
CRITICAL	**Dangerous Subprocess Use**	api\files.py:135	CWE-78	T1059	Detected subprocess function 'check_output' with user controlled data. A malicious actor could leverage this to perform
CRITICAL	**Subprocess Shell True**	api\files.py:135	CWE-78	T1059	Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn the command

SecureScope Security Report · OmarRao/analyzer Page 16 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
CRITICAL	**Dangerous System Call**	api\files.py:146	CWE-78	T1059	Found user-controlled data used in a system call. This could allow a malicious actor to execute commands. Use the 'subpr
CRITICAL	**Subprocess Injection**	api\files.py:156	CWE-78	T1059	Detected user input entering a `subprocess` call unsafely. This could result in a command injection vulnerability. An at
CRITICAL	**Dangerous Subprocess Use**	api\files.py:157	CWE-78	T1059	Detected subprocess function 'check_output' with user controlled data. A malicious actor could leverage this to perform
CRITICAL	**Subprocess Shell True**	api\files.py:157	CWE-78	T1059	Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn the command
CRITICAL	**Subprocess Injection**	api\files.py:166	CWE-78	T1059	Detected user input entering a `subprocess` call unsafely. This could result in a command injection vulnerability. An at
CRITICAL	**Dangerous Subprocess Use**	api\files.py:167	CWE-78	T1059	Detected subprocess function 'check_output' with user controlled data. A malicious actor could leverage this to perform
CRITICAL	**Subprocess Shell True**	api\files.py:167	CWE-78	T1059	Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn the command

SecureScope Security Report · OmarRao/analyzer Page 17 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
CRITICAL	**Tainted Sql String**	api\files.py:178	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\files.py:188	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Subprocess Injection**	api\files.py:191	CWE-78	T1059	Detected user input entering a `subprocess` call unsafely. This could result in a command injection vulnerability. An at
CRITICAL	**Dangerous Subprocess Use**	api\files.py:191	CWE-78	T1059	Detected subprocess function 'check_output' with user controlled data. A malicious actor could leverage this to perform
CRITICAL	**Subprocess Shell True**	api\files.py:191	CWE-78	T1059	Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn the command
CRITICAL	**Tainted Sql String**	api\files.py:192	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\files.py:202	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Subprocess Injection**	api\files.py:206	CWE-78	T1059	Detected user input entering a `subprocess` call unsafely. This could result in a command injection vulnerability. An at

SecureScope Security Report · OmarRao/analyzer Page 18 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
CRITICAL	**Dangerous Subprocess Use**	api\files.py:207	CWE-78	T1059	Detected subprocess function 'check_output' with user controlled data. A malicious actor could leverage this to perform
CRITICAL	**Subprocess Shell True**	api\files.py:207	CWE-78	T1059	Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn the command
CRITICAL	**Tainted Sql String**	api\loans.py:28	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\loans.py:38	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\loans.py:68	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\loans.py:79	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\loans.py:90	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\loans.py:92	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co

SecureScope Security Report · OmarRao/analyzer Page 19 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
CRITICAL	**Ssrf Injection Urllib**	api\loans.py:100	CWE-918	T1090	Data from request object is passed to a new server-side request. This could lead to a server-side request forgery (SSRF)
CRITICAL	**Ssrf Injection Urllib**	api\loans.py:101	CWE-918	T1090	Data from request object is passed to a new server-side request. This could lead to a server-side request forgery (SSRF)
CRITICAL	**Tainted Sql String**	api\loans.py:108	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\loans.py:119	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Subprocess Injection**	api\loans.py:130	CWE-78	T1059	Detected user input entering a `subprocess` call unsafely. This could result in a command injection vulnerability. An at
CRITICAL	**Dangerous Subprocess Use**	api\loans.py:131	CWE-78	T1059	Detected subprocess function 'check_output' with user controlled data. A malicious actor could leverage this to perform
CRITICAL	**Dangerous Subprocess Use**	api\loans.py:131	CWE-78	T1059	Detected subprocess function 'loan_statement' with user controlled data. A malicious actor could leverage this to perfor
CRITICAL	**Subprocess Shell True**	api\loans.py:132	CWE-78	T1059	Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn the command

SecureScope Security Report · OmarRao/analyzer Page 20 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
CRITICAL	**Tainted Sql String**	api\loans.py:144	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\loans.py:157	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\payments.py:33	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\payments.py:45	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Ssrf Injection Urllib**	api\payments.py:52	CWE-918	T1090	Data from request object is passed to a new server-side request. This could lead to a server-side request forgery (SSRF)
CRITICAL	**Ssrf Injection Urllib**	api\payments.py:53	CWE-918	T1090	Data from request object is passed to a new server-side request. This could lead to a server-side request forgery (SSRF)
CRITICAL	**Tainted Sql String**	api\payments.py:77	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Ssrf Injection Urllib**	api\payments.py:86	CWE-918	T1090	Data from request object is passed to a new server-side request. This could lead to a server-side request forgery (SSRF)

SecureScope Security Report · OmarRao/analyzer Page 21 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
CRITICAL	**Ssrf Injection Urllib**	api\payments.py:87	CWE-918	T1090	Data from request object is passed to a new server-side request. This could lead to a server-side request forgery (SSRF)
CRITICAL	**Tainted Sql String**	api\payments.py:106	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Ssrf Injection Urllib**	api\payments.py:114	CWE-918	T1090	Data from request object is passed to a new server-side request. This could lead to a server-side request forgery (SSRF)
CRITICAL	**Ssrf Injection Urllib**	api\payments.py:115	CWE-918	T1090	Data from request object is passed to a new server-side request. This could lead to a server-side request forgery (SSRF)
CRITICAL	**Ssrf Injection Urllib**	api\payments.py:116	CWE-918	T1090	Data from request object is passed to a new server-side request. This could lead to a server-side request forgery (SSRF)
CRITICAL	**Tainted Sql String**	api\payments.py:126	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Subprocess Injection**	api\payments.py:151	CWE-78	T1059	Detected user input entering a `subprocess` call unsafely. This could result in a command injection vulnerability. An at
CRITICAL	**Dangerous Subprocess Use**	api\payments.py:152	CWE-78	T1059	Detected subprocess function 'check_output' with user controlled data. A malicious actor could leverage this to perform

SecureScope Security Report · OmarRao/analyzer Page 22 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
CRITICAL	**Subprocess Shell True**	api\payments.py:152	CWE-78	T1059	Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn the command
CRITICAL	**Tainted Sql String**	api\payments.py:177	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\payments.py:205	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Dangerous System Call**	api\payments.py:221	CWE-78	T1059	Found user-controlled data used in a system call. This could allow a malicious actor to execute commands. Use the 'subpr
CRITICAL	**Tainted Sql String**	api\reports.py:31	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\reports.py:42	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Subprocess Injection**	api\reports.py:55	CWE-78	T1059	Detected user input entering a `subprocess` call unsafely. This could result in a command injection vulnerability. An at
CRITICAL	**Dangerous Subprocess Use**	api\reports.py:56	CWE-78	T1059	Detected subprocess function 'check_output' with user controlled data. A malicious actor could leverage this to perform

SecureScope Security Report · OmarRao/analyzer Page 23 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
CRITICAL	**Subprocess Shell True**	api\reports.py:58	CWE-78	T1059	Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn the command
CRITICAL	**Tainted Sql String**	api\reports.py:83	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Dangerous System Call**	api\reports.py:112	CWE-78	T1059	Found user-controlled data used in a system call. This could allow a malicious actor to execute commands. Use the 'subpr
CRITICAL	**Tainted Sql String**	api\reports.py:141	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Ssrf Injection Urllib**	api\reports.py:148	CWE-918	T1090	Data from request object is passed to a new server-side request. This could lead to a server-side request forgery (SSRF)
CRITICAL	**Ssrf Injection Urllib**	api\reports.py:149	CWE-918	T1090	Data from request object is passed to a new server-side request. This could lead to a server-side request forgery (SSRF)
CRITICAL	**Tainted Sql String**	api\reports.py:162	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Ssrf Injection Urllib**	api\reports.py:171	CWE-918	T1090	Data from request object is passed to a new server-side request. This could lead to a server-side request forgery (SSRF)

SecureScope Security Report · OmarRao/analyzer Page 24 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
CRITICAL	**Ssrf Injection Urllib**	api\reports.py:172	CWE-918	T1090	Data from request object is passed to a new server-side request. This could lead to a server-side request forgery (SSRF)
CRITICAL	**Tainted Sql String**	api\reports.py:182	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Subprocess Injection**	api\reports.py:190	CWE-78	T1059	Detected user input entering a `subprocess` call unsafely. This could result in a command injection vulnerability. An at
CRITICAL	**Dangerous Subprocess Use**	api\reports.py:191	CWE-78	T1059	Detected subprocess function 'check_output' with user controlled data. A malicious actor could leverage this to perform
CRITICAL	**Dangerous Subprocess Use**	api\reports.py:191	CWE-78	T1059	Detected subprocess function 'render_template' with user controlled data. A malicious actor could leverage this to perfo
CRITICAL	**Subprocess Shell True**	api\reports.py:192	CWE-78	T1059	Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn the command
CRITICAL	**Tainted Sql String**	api\transactions.py:30	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\transactions.py:45	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co

SecureScope Security Report · OmarRao/analyzer Page 25 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
CRITICAL	**Tainted Sql String**	api\transactions.py:60	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\transactions.py:61	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\transactions.py:64	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\transactions.py:65	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Ssrf Injection Urllib**	api\transactions.py:76	CWE-918	T1090	Data from request object is passed to a new server-side request. This could lead to a server-side request forgery (SSRF)
CRITICAL	**Ssrf Injection Urllib**	api\transactions.py:77	CWE-918	T1090	Data from request object is passed to a new server-side request. This could lead to a server-side request forgery (SSRF)
CRITICAL	**Ssrf Injection Urllib**	api\transactions.py:78	CWE-918	T1090	Data from request object is passed to a new server-side request. This could lead to a server-side request forgery (SSRF)
CRITICAL	**Ssrf Injection Urllib**	api\transactions.py:79	CWE-918	T1090	Data from request object is passed to a new server-side request. This could lead to a server-side request forgery (SSRF)

SecureScope Security Report · OmarRao/analyzer Page 26 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
CRITICAL	**Tainted Sql String**	api\transactions.py:96	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Subprocess Injection**	api\transactions.py:113	CWE-78	T1059	Detected user input entering a `subprocess` call unsafely. This could result in a command injection vulnerability. An at
CRITICAL	**Dangerous Subprocess Use**	api\transactions.py:114	CWE-78	T1059	Detected subprocess function 'check_output' with user controlled data. A malicious actor could leverage this to perform
CRITICAL	**Dangerous Subprocess Use**	api\transactions.py:114	CWE-78	T1059	Detected subprocess function 'get_receipt' with user controlled data. A malicious actor could leverage this to perform c
CRITICAL	**Subprocess Shell True**	api\transactions.py:114	CWE-78	T1059	Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn the command
CRITICAL	**Dangerous System Call**	api\transactions.py:133	CWE-78	T1059	Found user-controlled data used in a system call. This could allow a malicious actor to execute commands. Use the 'subpr
CRITICAL	**Tainted Sql String**	api\transactions.py:155	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co

SecureScope Security Report · OmarRao/analyzer Page 27 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
CRITICAL	**Tainted Sql String**	api\transactions.py:156	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\transactions.py:158	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\transactions.py:168	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\transactions.py:180	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\transactions.py:202	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\transactions.py:213	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Subprocess Injection**	api\transactions.py:224	CWE-78	T1059	Detected user input entering a `subprocess` call unsafely. This could result in a command injection vulnerability. An at
CRITICAL	**Dangerous Subprocess Use**	api\transactions.py:225	CWE-78	T1059	Detected subprocess function 'check_output' with user controlled data. A malicious actor could leverage this to perform

SecureScope Security Report · OmarRao/analyzer Page 28 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
CRITICAL	**Subprocess Shell True**	api\transactions.py:225	CWE-78	T1059	Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn the command
CRITICAL	**Ssrf Injection Urllib**	api\transactions.py:232	CWE-918	T1090	Data from request object is passed to a new server-side request. This could lead to a server-side request forgery (SSRF)
CRITICAL	**Ssrf Injection Urllib**	api\transactions.py:233	CWE-918	T1090	Data from request object is passed to a new server-side request. This could lead to a server-side request forgery (SSRF)
CRITICAL	**Ssrf Injection Urllib**	api\transactions.py:234	CWE-918	T1090	Data from request object is passed to a new server-side request. This could lead to a server-side request forgery (SSRF)
CRITICAL	**Ssrf Injection Urllib**	api\transactions.py:235	CWE-918	T1090	Data from request object is passed to a new server-side request. This could lead to a server-side request forgery (SSRF)
CRITICAL	**Tainted Sql String**	api\transactions.py:248	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\transactions.py:262	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\transactions.py:266	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co

SecureScope Security Report · OmarRao/analyzer Page 29 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
CRITICAL	**Tainted Sql String**	api\users.py:30	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\users.py:42	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\users.py:56	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\users.py:60	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\users.py:74	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Subprocess Injection**	api\users.py:90	CWE-78	T1059	Detected user input entering a `subprocess` call unsafely. This could result in a command injection vulnerability. An at
CRITICAL	**Dangerous Subprocess Use**	api\users.py:91	CWE-78	T1059	Detected subprocess function 'check_output' with user controlled data. A malicious actor could leverage this to perform
CRITICAL	**Dangerous Subprocess Use**	api\users.py:91	CWE-78	T1059	Detected subprocess function 'export_user_data' with user controlled data. A malicious actor could leverage this to perf

SecureScope Security Report · OmarRao/analyzer Page 30 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
CRITICAL	**Subprocess Shell True**	api\users.py:91	CWE-78	T1059	Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn the command
CRITICAL	**Tainted Sql String**	api\users.py:126	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\users.py:135	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\users.py:145	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\users.py:155	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\users.py:158	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\users.py:167	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\users.py:170	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co

SecureScope Security Report · OmarRao/analyzer Page 31 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
CRITICAL	**Tainted Sql String**	api\users.py:188	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\users.py:210	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	api\users.py:219	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Dangerous System Call**	api\users.py:229	CWE-78	T1059	Found user-controlled data used in a system call. This could allow a malicious actor to execute commands. Use the 'subpr
CRITICAL	**Tainted Sql String**	app.py:72	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	app.py:111	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Subprocess Injection**	app.py:125	CWE-78	T1059	Detected user input entering a `subprocess` call unsafely. This could result in a command injection vulnerability. An at
CRITICAL	**Dangerous Subprocess Use**	app.py:125	CWE-78	T1059	Detected subprocess function 'check_output' with user controlled data. A malicious actor could leverage this to perform

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
CRITICAL	**Subprocess Shell True**	app.py:125	CWE-78	T1059	Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn the command
CRITICAL	**Dangerous System Call**	app.py:145	CWE-78	T1059	Found user-controlled data used in a system call. This could allow a malicious actor to execute commands. Use the 'subpr
CRITICAL	**Tainted Sql String**	app.py:155	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	app.py:175	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	app.py:177	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	app.py:178	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	app.py:179	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	app.py:206	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co

SecureScope Security Report · OmarRao/analyzer Page 33 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
CRITICAL	**Ssrf Injection Urllib**	app.py:218	CWE-918	T1090	Data from request object is passed to a new server-side request. This could lead to a server-side request forgery (SSRF)
CRITICAL	**Subprocess Shell True**	jobs\scheduled.py:26	CWE-78	T1059	Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn the command
CRITICAL	**Subprocess Shell True**	jobs\scheduled.py:48	CWE-78	T1059	Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn the command
CRITICAL	**Subprocess Shell True**	jobs\scheduled.py:69	CWE-78	T1059	Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn the command
CRITICAL	**Subprocess Shell True**	jobs\scheduled.py:111	CWE-78	T1059	Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn the command
CRITICAL	**Subprocess Shell True**	jobs\scheduled.py:139	CWE-78	T1059	Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn the command
CRITICAL	**Subprocess Shell True**	jobs\scheduled.py:150	CWE-78	T1059	Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn the command
CRITICAL	**Tainted Sql String**	middleware\auth.py:31	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co

SecureScope Security Report · OmarRao/analyzer Page 34 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
CRITICAL	**Tainted Sql String**	middleware\auth.py:52	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Tainted Sql String**	middleware\auth.py:68	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	**Subprocess Shell True**	services\crypto_service.py:158	CWE-78	T1059	Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn the command
CRITICAL	**Subprocess Shell True**	services\crypto_service.py:167	CWE-78	T1059	Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn the command
CRITICAL	**Subprocess Shell True**	services\email.py:47	CWE-78	T1059	Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn the command
CRITICAL	**Subprocess Shell True**	services\email.py:140	CWE-78	T1059	Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn the command
CRITICAL	**Subprocess Shell True**	services\email.py:152	CWE-78	T1059	Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn the command
CRITICAL	**Subprocess Shell True**	services\email.py:169	CWE-78	T1059	Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn the command

SecureScope Security Report · OmarRao/analyzer Page 35 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
CRITICAL	**Subprocess Shell True**	services\logger.py:67	CWE-78	T1059	Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn the command
CRITICAL	**Subprocess Shell True**	services\logger.py:144	CWE-78	T1059	Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn the command
CRITICAL	**Subprocess Shell True**	services\logger.py:152	CWE-78	T1059	Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn the command
CRITICAL	**Subprocess Shell True**	services\notification.py:132	CWE-78	T1059	Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn the command
CRITICAL	**Subprocess Shell True**	services\search.py:150	CWE-78	T1059	Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn the command
CRITICAL	**Subprocess Shell True**	utils\formatters.py:86	CWE-78	T1059	Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn the command
CRITICAL	**Subprocess Shell True**	utils\formatters.py:144	CWE-78	T1059	Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn the command
CRITICAL	**Subprocess Shell True**	utils\validators.py:157	CWE-78	T1059	Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn the command

SecureScope Security Report · OmarRao/analyzer Page 36 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
WARNING	**Sql Injection Db Cursor Execute**	api\accounts.py:36	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\accounts.py:91	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\accounts.py:110	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\accounts.py:139	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\accounts.py:140	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\accounts.py:141	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\accounts.py:154	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\accounts.py:155	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected

SecureScope Security Report · OmarRao/analyzer Page 37 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
WARNING	**Sql Injection Db Cursor Execute**	api\accounts.py:186	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\accounts.py:196	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\accounts.py:244	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\accounts.py:245	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Directly Returned Format String**	api\accounts.py:252	CWE-79	T1059.007	Detected Flask route directly returning a formatted string. This is subject to cross-site scripting if user input can re
WARNING	**Raw Html Format**	api\accounts.py:252	CWE-79	T1059.007	Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure methods of
WARNING	**Raw Html Format**	api\accounts.py:252	CWE-79	T1059.007	Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure methods of
WARNING	**Sql Injection Db Cursor Execute**	api\admin.py:48	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected

SecureScope Security Report · OmarRao/analyzer Page 38 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
WARNING	**Sql Injection Db Cursor Execute**	api\admin.py:58	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\admin.py:67	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\admin.py:68	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\admin.py:123	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\admin.py:124	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\admin.py:165	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\admin.py:166	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Directly Returned Format String**	api\admin.py:178	CWE-79	T1059.007	Detected Flask route directly returning a formatted string. This is subject to cross-site scripting if user input can re

SecureScope Security Report · OmarRao/analyzer Page 39 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
WARNING	**Raw Html Format**	api\admin.py:178	CWE-79	T1059.007	Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure methods of
WARNING	**Raw Html Format**	api\admin.py:178	CWE-79	T1059.007	Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure methods of
WARNING	**Sql Injection Db Cursor Execute**	api\admin.py:195	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\admin.py:249	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\admin.py:250	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\admin.py:251	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\auth.py:30	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Insecure Hash Algorithm Md5**	api\auth.py:36	CWE-327	T1600	Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is therefore not suitable a

SecureScope Security Report · OmarRao/analyzer Page 40 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
WARNING	**Md5 Used As Password**	api\auth.py:36	CWE-327	T1600	It looks like MD5 is used as a password hash. MD5 is not considered a secure password hash because it can be cracked by
WARNING	**Sql Injection Db Cursor Execute**	api\auth.py:55	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\auth.py:56	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Insecure Hash Algorithm Md5**	api\auth.py:68	CWE-327	T1600	Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is therefore not suitable a
WARNING	**Md5 Used As Password**	api\auth.py:68	CWE-327	T1600	It looks like MD5 is used as a password hash. MD5 is not considered a secure password hash because it can be cracked by
WARNING	**Sql Injection Db Cursor Execute**	api\auth.py:82	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\auth.py:83	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\auth.py:83	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected

SecureScope Security Report · OmarRao/analyzer Page 41 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
WARNING	**Sql Injection Db Cursor Execute**	api\auth.py:98	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\auth.py:110	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\auth.py:111	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Insecure Hash Algorithm Md5**	api\auth.py:120	CWE-327	T1600	Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is therefore not suitable a
WARNING	**Md5 Used As Password**	api\auth.py:120	CWE-327	T1600	It looks like MD5 is used as a password hash. MD5 is not considered a secure password hash because it can be cracked by
WARNING	**Sql Injection Db Cursor Execute**	api\auth.py:129	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\auth.py:129	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Insecure Hash Algorithm Md5**	api\auth.py:133	CWE-327	T1600	Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is therefore not suitable a

SecureScope Security Report · OmarRao/analyzer Page 42 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
WARNING	**Md5 Used As Password**	api\auth.py:133	CWE-327	T1600	It looks like MD5 is used as a password hash. MD5 is not considered a secure password hash because it can be cracked by
WARNING	**Insecure Hash Algorithm Md5**	api\auth.py:138	CWE-327	T1600	Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is therefore not suitable a
WARNING	**Md5 Used As Password**	api\auth.py:138	CWE-327	T1600	It looks like MD5 is used as a password hash. MD5 is not considered a secure password hash because it can be cracked by
WARNING	**Sql Injection Db Cursor Execute**	api\auth.py:147	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\auth.py:188	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\auth.py:188	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\auth.py:202	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\auth.py:216	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected

SecureScope Security Report · OmarRao/analyzer Page 43 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
WARNING	**Sql Injection Db Cursor Execute**	api\auth.py:229	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\files.py:82	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\files.py:96	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\files.py:97	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Path Traversal Open**	api\files.py:109	CWE-22	T1083	Found request data in a call to 'open'. Ensure the request data is validated or sanitized, otherwise it could result in
WARNING	**Directly Returned Format String**	api\files.py:115	CWE-79	T1059.007	Detected Flask route directly returning a formatted string. This is subject to cross-site scripting if user input can re
WARNING	**Raw Html Format**	api\files.py:115	CWE-79	T1059.007	Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure methods of
WARNING	**Raw Html Format**	api\files.py:115	CWE-79	T1059.007	Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure methods of

SecureScope Security Report · OmarRao/analyzer Page 44 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
WARNING	**Sql Injection Db Cursor Execute**	api\files.py:174	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\files.py:175	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\files.py:199	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\loans.py:24	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\loans.py:77	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\loans.py:86	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\loans.py:86	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\loans.py:87	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected

SecureScope Security Report · OmarRao/analyzer Page 45 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
WARNING	**Sql Injection Db Cursor Execute**	api\loans.py:100	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\loans.py:115	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\loans.py:153	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\loans.py:154	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\payments.py:28	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\payments.py:100	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\reports.py:26	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\reports.py:120	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected

SecureScope Security Report · OmarRao/analyzer Page 46 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
WARNING	**Sql Injection Db Cursor Execute**	api\reports.py:121	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Raw Html Format**	api\reports.py:127	CWE-79	T1059.007	Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure methods of
WARNING	**Raw Html Format**	api\reports.py:127	CWE-79	T1059.007	Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure methods of
WARNING	**Raw Html Format**	api\reports.py:129	CWE-79	T1059.007	Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure methods of
WARNING	**Raw Html Format**	api\reports.py:129	CWE-79	T1059.007	Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure methods of
WARNING	**Directly Returned Format String**	api\reports.py:131	CWE-79	T1059.007	Detected Flask route directly returning a formatted string. This is subject to cross-site scripting if user input can re
WARNING	**Sql Injection Db Cursor Execute**	api\reports.py:136	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\reports.py:137	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected

SecureScope Security Report · OmarRao/analyzer Page 47 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
WARNING	**Sql Injection Db Cursor Execute**	api\reports.py:138	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\reports.py:158	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\reports.py:159	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Directly Returned Format String**	api\reports.py:166	CWE-79	T1059.007	Detected Flask route directly returning a formatted string. This is subject to cross-site scripting if user input can re
WARNING	**Raw Html Format**	api\reports.py:166	CWE-79	T1059.007	Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure methods of
WARNING	**Raw Html Format**	api\reports.py:166	CWE-79	T1059.007	Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure methods of
WARNING	**Sql Injection Db Cursor Execute**	api\transactions.py:53	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\transactions.py:53	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected

SecureScope Security Report · OmarRao/analyzer Page 48 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
WARNING	**Sql Injection Db Cursor Execute**	api\transactions.py:54	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\transactions.py:54	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\transactions.py:55	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\transactions.py:55	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\transactions.py:166	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\transactions.py:175	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\transactions.py:176	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\transactions.py:177	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected

SecureScope Security Report · OmarRao/analyzer Page 49 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
WARNING	**Sql Injection Db Cursor Execute**	api\transactions.py:201	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\transactions.py:244	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\transactions.py:245	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Directly Returned Format String**	api\transactions.py:252	CWE-79	T1059.007	Detected Flask route directly returning a formatted string. This is subject to cross-site scripting if user input can re
WARNING	**Raw Html Format**	api\transactions.py:252	CWE-79	T1059.007	Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure methods of
WARNING	**Raw Html Format**	api\transactions.py:252	CWE-79	T1059.007	Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure methods of
WARNING	**Sql Injection Db Cursor Execute**	api\users.py:38	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\users.py:39	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected

SecureScope Security Report · OmarRao/analyzer Page 50 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
WARNING	**Directly Returned Format String**	api\users.py:44	CWE-79	T1059.007	Detected Flask route directly returning a formatted string. This is subject to cross-site scripting if user input can re
WARNING	**Raw Html Format**	api\users.py:44	CWE-79	T1059.007	Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure methods of
WARNING	**Raw Html Format**	api\users.py:44	CWE-79	T1059.007	Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure methods of
WARNING	**Sql Injection Db Cursor Execute**	api\users.py:51	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\users.py:52	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\users.py:53	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Insecure Hash Algorithm Md5**	api\users.py:111	CWE-327	T1600	Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is therefore not suitable a
WARNING	**Md5 Used As Password**	api\users.py:111	CWE-327	T1600	It looks like MD5 is used as a password hash. MD5 is not considered a secure password hash because it can be cracked by

SecureScope Security Report · OmarRao/analyzer Page 51 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
WARNING	**Sql Injection Db Cursor Execute**	api\users.py:122	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\users.py:123	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Directly Returned Format String**	api\users.py:129	CWE-79	T1059.007	Detected Flask route directly returning a formatted string. This is subject to cross-site scripting if user input can re
WARNING	**Raw Html Format**	api\users.py:129	CWE-79	T1059.007	Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure methods of
WARNING	**Raw Html Format**	api\users.py:129	CWE-79	T1059.007	Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure methods of
WARNING	**Sql Injection Db Cursor Execute**	api\users.py:143	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\users.py:153	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	api\users.py:166	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected

SecureScope Security Report · OmarRao/analyzer Page 52 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
WARNING	**Sql Injection Db Cursor Execute**	api\users.py:216	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	app.py:67	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	app.py:68	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	app.py:105	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Directly Returned Format String**	app.py:116	CWE-79	T1059.007	Detected Flask route directly returning a formatted string. This is subject to cross-site scripting if user input can re
WARNING	**Raw Html Format**	app.py:116	CWE-79	T1059.007	Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure methods of
WARNING	**Raw Html Format**	app.py:116	CWE-79	T1059.007	Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure methods of
WARNING	**Directly Returned Format String**	app.py:126	CWE-79	T1059.007	Detected Flask route directly returning a formatted string. This is subject to cross-site scripting if user input can re

SecureScope Security Report · OmarRao/analyzer Page 53 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
WARNING	**Raw Html Format**	app.py:126	CWE-79	T1059.007	Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure methods of
WARNING	**Raw Html Format**	app.py:126	CWE-79	T1059.007	Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure methods of
WARNING	**Path Traversal Open**	app.py:132	CWE-22	T1083	Found request data in a call to 'open'. Ensure the request data is validated or sanitized, otherwise it could result in
WARNING	**Directly Returned Format String**	app.py:136	CWE-79	T1059.007	Detected Flask route directly returning a formatted string. This is subject to cross-site scripting if user input can re
WARNING	**Raw Html Format**	app.py:136	CWE-79	T1059.007	Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure methods of
WARNING	**Raw Html Format**	app.py:136	CWE-79	T1059.007	Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure methods of
WARNING	**Sql Injection Db Cursor Execute**	app.py:170	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	app.py:170	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected

SecureScope Security Report · OmarRao/analyzer Page 54 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
WARNING	**Sql Injection Db Cursor Execute**	app.py:171	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Sql Injection Db Cursor Execute**	app.py:199	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	**Insecure Hash Algorithm Md5**	app.py:203	CWE-327	T1600	Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is therefore not suitable a
WARNING	**Avoid_App_Run_With_Bad_Host**	app.py:226	CWE-668		Running flask app with host 0.0.0.0 could expose the server publicly.
WARNING	**Debug Enabled**	app.py:226	CWE-489		Detected Flask app with debug=True. Do not deploy to production with this flag enabled as it will leak sensitive informa
WARNING	**Insecure Hash Algorithm Md5**	middleware\auth.py:139	CWE-327	T1600	Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is therefore not suitable a
WARNING	**Insecure Hash Algorithm Md5**	models.py:59	CWE-327	T1600	Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is therefore not suitable a
WARNING	**Md5 Used As Password**	models.py:59	CWE-327	T1600	It looks like MD5 is used as a password hash. MD5 is not considered a secure password hash because it can be cracked by

SecureScope Security Report · OmarRao/analyzer Page 55 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
WARNING	**Insecure Hash Algorithm Md5**	models.py:79	CWE-327	T1600	Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is therefore not suitable a
WARNING	**Md5 Used As Password**	models.py:79	CWE-327	T1600	It looks like MD5 is used as a password hash. MD5 is not considered a secure password hash because it can be cracked by
WARNING	**Insecure Hash Algorithm Md5**	services\crypto_service.py:26	CWE-327	T1600	Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is therefore not suitable a
WARNING	**Md5 Used As Password**	services\crypto_service.py:26	CWE-327	T1600	It looks like MD5 is used as a password hash. MD5 is not considered a secure password hash because it can be cracked by
WARNING	**Insecure Hash Algorithm Sha1**	services\crypto_service.py:31	CWE-327	T1600	Detected SHA1 hash algorithm which is considered insecure. SHA1 is not collision resistant and is therefore not suitable
WARNING	**Insecure Hash Algorithm Md5**	services\crypto_service.py:36	CWE-327	T1600	Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is therefore not suitable a
WARNING	**Insecure Hash Algorithm Md5**	services\crypto_service.py:51	CWE-327	T1600	Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is therefore not suitable a

SecureScope Security Report · OmarRao/analyzer Page 56 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
WARNING	**Insecure Hash Algorithm Md5**	services\crypto_service.py:77	CWE-327	T1600	Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is therefore not suitable a
WARNING	**Insecure Hash Algorithm Md5**	services\crypto_service.py:82	CWE-327	T1600	Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is therefore not suitable a
WARNING	**Insecure Hash Algorithm Md5**	services\crypto_service.py:88	CWE-327	T1600	Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is therefore not suitable a
WARNING	**Insecure Hash Algorithm Md5**	services\crypto_service.py:96	CWE-327	T1600	Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is therefore not suitable a
WARNING	**Insecure Hash Algorithm Md5**	services\crypto_service.py:141	CWE-327	T1600	Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is therefore not suitable a
WARNING	**Insecure Hash Algorithm Md5**	services\crypto_service.py:151	CWE-327	T1600	Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is therefore not suitable a
WARNING	**Md5 Used As Password**	services\crypto_service.py:151	CWE-327	T1600	It looks like MD5 is used as a password hash. MD5 is not considered a secure password hash because it can be cracked by

SecureScope Security Report · OmarRao/analyzer Page 57 of 62

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
WARNING	**Insecure Hash Algorithm Md5**	utils.py:13	CWE-327	T1600	Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is therefore not suitable a
WARNING	**Md5 Used As Password**	utils.py:13	CWE-327	T1600	It looks like MD5 is used as a password hash. MD5 is not considered a secure password hash because it can be cracked by
WARNING	**Insecure Hash Algorithm Md5**	utils.py:43	CWE-327	T1600	Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is therefore not suitable a

SecureScope Security Report · OmarRao/analyzer Page 58 of 62

RANSOMWARE INTELLIGENCE

Ransomware & APT Threat Assessment

Static analysis findings analysed against 11 known ransomware behavioral patterns, 9 threat actor families, and 12 active CVEs exploited in ransomware campaigns. Blast radius estimates the maximum damage potential if an attacker exploited all detected vulnerabilities.

RANSOMWARE RISK SCORE

100/100

HIGH RISK

BLAST RADIUS

100/100

CATASTROPHIC

Full infrastructure compromise. Data encryption, exfiltration, and unrecoverable destructi...

APT ATTRIBUTION

APT DETECTED

Confidence: **81%**

TA505 / FIN11

RU Russia / Ukraine

DETECTED RANSOMWARE BEHAVIORS

	BEHAVIOR	MITRE	DESCRIPTION	SEVERITY	HITS
🔪	**Security Process Kill**	T1562.001	Terminates AV/EDR/backup processes before encryption	CRITICAL	103
⚡	**Command Injection (Execution)**	T1059	OS command injection enables direct payload execution	CRITICAL	103
🔐	**File Encryption**	T1486	Reads and re-encrypts files using symmetric/asymmetric crypto	CRITICAL	36
🔑	**Credential Harvesting**	T1552	Extracts credentials from memory, files, or environment variables	CRITICAL	11
📌	**Hardcoded Credentials**	T1552.001	Hardcoded secrets enable attackers to authenticate and deploy ransomwa	CRITICAL	11
💉	**SQL Injection (Initial Access)**	T1190	SQLi can be used for initial access or credential theft to deliver ran	HIGH	287
🌐	**Lateral Movement**	T1021	Spreads to other systems via network shares, RDP, or SMB	HIGH	103
🔓	**Weak Cryptographic Keys**	T1600	Uses weak algorithms (MD5/SHA1/ECB) that ransomware can exploit for ke	HIGH	33
📤	**Data Exfiltration**	T1041	Uploads sensitive data to remote C2 before encryption	HIGH	24

MATCHED THREAT ACTOR FAMILIES

SecureScope Security Report · OmarRao/analyzer Page 59 of 62

ClOp (TA505 · FIN11 · Lace Tempest)

RU Russia / Ukraine | ACTIVE | Big Game Hunting / RaaS

93% match confidence

Pioneered mass-exploitation of file-transfer platforms. MOVEit attack compromised 2000+ organisations globally. Focuses on data theft over encryption.

SECTORS	KNOWN VICTIMS	MATCHED CVES
Finance, Healthcare, Education, Government	MOVEit users (2000+ orgs) · GoAnywhere users	CVE-2023-34362, CVE-2023-0669

Matched Behaviors: file encryption, file enumeration, data exfiltration, ssrf, sql injection

DarkSide (Carbon Spider · Gold Dupont)

RU Russia / Eastern Europe | REBRANDED | Ransomware-as-a-Service (RaaS)

93% match confidence

Colonial Pipeline attack (2021) caused fuel shortages across US East Coast. Group claimed to 'only target companies that can pay' — later rebranded as BlackMatter.

SECTORS	KNOWN VICTIMS	MATCHED CVES
Energy, Oil & Gas, Finance, Manufacturing	Colonial Pipeline · Brenntag	CVE-2021-20016, CVE-2019-0708

Matched Behaviors: file encryption, data exfiltration, credential theft, lateral movement, process termination

LockBit 3.0 (LockBit Black · LockBit 2.0)

RU Russia / Eastern Europe | ACTIVE | Ransomware-as-a-Service (RaaS)

85% match confidence

Most prolific ransomware group globally. Uses triple extortion — encryption, data leak, DDoS. Known for fastest encryption speed via multi-threading.

SECTORS	KNOWN VICTIMS	MATCHED CVES
Healthcare, Finance, Government, Manufacturing	Boeing · ICBC	CVE-2023-4966, CVE-2021-44228, CVE-2023-0669

Matched Behaviors: file encryption, file enumeration, process termination, lateral movement, credential theft

ACTIVE CVE VARIANTS EXPLOITED

CVE ID	VULNERABILITY	DESCRIPTION	CVSS	FAMILY
CVE-2021-44228	**Log4Shell**	Apache Log4j2 RCE, exploited by ransomware groups for initial access	10.0	LockBit 3.0
CVE-2020-1472	**Zerologon**	Netlogon privilege escalation — instant domain compromise	10.0	Conti
CVE-2023-34362	**MOVEit SQLi**	MOVEit Transfer SQL injection — ClOp mass exploitation campaign	9.8	ClOp
CVE-2021-20016	**SonicWall SSL-VPN**	SonicWall SSL-VPN SQL injection used by DarkSide for initial access	9.8	DarkSide
CVE-2019-0708	**BlueKeep**	RDP wormable RCE — unauthenticated remote code execution	9.8	DarkSide

SecureScope Security Report · OmarRao/analyzer Page 60 of 62

CVE ID	VULNERABILITY	DESCRIPTION	CVSS	FAMILY
CVE-2021-26855	**ProxyLogon**	Microsoft Exchange pre-auth RCE — ransomware initial access vector	9.8	Conti
CVE-2022-47966	**ManageEngine RCE**	Zoho ManageEngine pre-auth RCE used by North Korean APT	9.8	Lazarus Group
CVE-2021-30116	**Kaseya VSA**	Kaseya VSA credential leak + RCE — REvil supply chain attack	9.8	REvil / Sodinokibi

AFFECTED CODE SECTIONS

FILE	MATCHED BEHAVIORS	COUNT
api\accounts.py	Command Injection (Execution), Data Exfiltration, Lateral Movement	22
api\auth.py	Credential Harvesting, Data Exfiltration, File Encryption	21
api\loans.py	Data Exfiltration, SSRF (C2 Reachback)	4
api\admin.py	File Encryption, File Enumeration	2
api\payments.py	Data Exfiltration, SSRF (C2 Reachback)	2
api\files.py	File Enumeration	1
app.py	File Enumeration	1

SecureScope Security Report · OmarRao/analyzer Page 61 of 62

METHODOLOGY

How This Report Was Generated

ANALYSIS ENGINE	
Static Analysis	Semgrep OSS
Rulesets	OWASP, CWE-25, Secrets
Threat Framework	MITRE ATT&CK v14
CVE Audit	pip-audit / npm audit

SCORING MODEL	
Critical weight	× 10 per finding
Warning weight	× 3 per finding
CVE weight	× 8 per CVE
Score cap	100

RESPONSIBLE DISCLOSURE

This report is generated automatically by SecureScope and is intended for the repository owner and their authorised security team. Findings reflect the state of the codebase at the time of analysis. They should be validated by a qualified security engineer before remediation work begins.

SecureScope maps findings to MITRE ATT&CK v14 and CWE identifiers. These mappings are best-effort and should not be treated as a definitive classification. False positives are possible — Semgrep static analysis is pattern-based and does not perform dynamic execution.

For questions about this report or the SecureScope platform, refer to the project repository at github.com/OmarRao/secure-scope.

SecureScope BETA

AI-Powered GitHub Security Analysis · MITRE ATT&CK v14 · CWE Top 25

Omar Rao
Engineer by trait
Data Resilience, Privacy & Cybersecurity Expert
Founder, SecureScope

SecureScope Security Report · OmarRao/analyzer · Page 62 of 62